Современные технологии сборки, линтинга и роутинга во фронтенд-приложениях

Vite · Oxidation Compiler · TanStack Router

Лекция 1

Почему старый подход устарел

- Dev-сервер сначала собирает весь проект в один бандл
- Время старта растёт вместе с кодовой базой
- Правка файла запускает пересборку цепочки модулей
- HMR теряет состояние или перезагружает страницу

> Узкое место — не код, а обязательная сборка перед каждым запуском

Как работает Vite

- Браузер понимает ES-модули нативно — import/export без сборщика
- Dev-сервер отдаёт файлы по запросу, а не собирает бандл заранее
- node_modules предсобираются один раз (esbuild): CommonJS → ESM
- Правка модуля инвалидирует только его цепочку, а не проект

> Dev = ESM + on-demand transform. Build = Rollup/Rolldown + minify + tree-shaking

Два режима работы

Параметр	Dev — vp run dev	Build — vp run build
Цель	мгновенный цикл правок	готовый бандл
Модули	ESM по запросу	чанки и ассеты
Зависимости	esbuild prebundle	Rollup / Rolldown
Скорость	старт почти мгновенный	секунды и минуты
Оптимизация	нет	minify, tree-shaking

Старт проекта

- npm create vite@latest my-app -- --template react-ts

```
npm create vite@latest my-app -- --template react-ts
cd my-app
npm install
npm run dev
```

```
vp install
vp run dev      # dev-сервер на :3000
vp run build    # tsc -b && сборка
```

predev и prebuild автоматически перегенерируют API-клиенты, роуты и события

Анатомия проекта

```
my-app/
├─ index.html
├─ vite.config.ts
├─ package.json
├─ public/
│  └─ favicon.svg
└─ src/
   ├─ main.tsx
   └─ App.tsx
```

- index.html — точка входа
- module-скрипт запускает граф модулей
- vite.config.ts — один конфиг на всё
- public/ — файлы без обработки
- src/ — код приложения

vite.config.ts: наш конфиг

```
export default defineConfig({
  plugins: lazyPlugins(() => [tanstackRouter({ target: "react" }), react()]),
  resolve: { alias: { "@": resolve(__dirname, "./src") } },
  css: { preprocessorOptions: { scss: { additionalData: `@use "@/styles/variables.scss" as *;` } } },
  server: {
    port: 3000,
    proxy: {
      "/api":  { target: "http://localhost:3001", changeOrigin: true },
      "/mqtt": { target: "ws://localhost:8083", ws: true },
    },
  },
});
```

Один файл описывает плагины, алиасы, препроцессор и прокси

Ключевые опции конфига

- root — папка с index.html (по умолч. process.cwd())
- base — префикс путей и BASE_URL (по умолч. '/')
- publicDir — статика, копируется в dist (по умолч. 'public')
- define — замена идентификаторов на этапе сборки
- envDir / envPrefix — откуда и с каким префиксом грузить .env
- resolve.extensions — порядок расширений при import без расширения

```
define: { __APP_VERSION__: JSON.stringify('1.0.0') },
envPrefix: 'VITE_',
```

Конфиг как функция и loadEnv

```
export default defineConfig(({ command, mode }) => {
  const env = loadEnv(mode, process.cwd(), '');
  return {
    plugins: command === 'build' ? [legacy()] : [],
    define: { __API__: JSON.stringify(env.VITE_API_URL) },
  };
});
```

- .env недоступны в vite.config.ts через process.env — Vite грузит их после конфига
- loadEnv(mode, root, prefix) читает .env вручную
- command = 'serve' (dev) или 'build'
- mode задаётся флагом --mode

Опции dev-сервера

Опция	Что делает
port (5173)	порт dev-сервера
strictPort	не занимать другой порт, если занят — ошибка
https	TLS/HTTP2; плагин @vitejs/plugin-basic-ssl
open	открыть браузер при старте (строка = pathname)
cors	true = любой origin, либо объект настроек
forwardConsole	переносить console браузера в терминал (для AI-агентов)

- server.proxy перенаправляет запросы на бэкенд, обходя CORS
- changeOrigin подменяет Host; ws: true для WebSocket

Ассеты: импорт и public/

```
import imgUrl from './img.png';
// imgUrl === '/assets/img-abc123.png' (URL)

<img src={imgUrl} alt="..." />
```

- Импортированная картинка → URL с хешем, попадает в бандл
- Мелкие файлы (≤ 4 КБ) инлайнятся как data URL
- public/ — файлы отдадаются как есть, без хеша и обработки
- В prod public/ копируется в корень dist

Env-переменные и режимы

- import.meta.env — подстановка на этапе сборки
- В клиент попадают только переменные с префиксом VITE_
- Кастомные всегда string, встроенные DEV/PROD/SSR — boolean
- Порядок: .env.[mode].local → .env.[mode] → .env.local → .env
- Типизация — в src/vite-env.d.ts через ImportMetaEnv

```
// .env.development
VITE_API_URL=/api
VITE_MOCK_API=true
```

Pre-bundling зависимостей

- Vite предсобирает node_modules один раз (esbuild)
- Результат лежит в node_modules/.vite
- Кеш инвалидируется по package.json и lock-файлу
- --force сбросит кеш при проблемах

```
vite --force   # пересобрать кеш зависимостей
```

> Без pre-bundle каждый import из node_modules превращался бы в сотни запросов

HMR: обновляем только то, что изменилось

- Vite хранит граф модулей и их импортёров
- Изменение уходит в браузер по WebSocket
- Обновляется ближайшая HMR-граница, а не страница
- React Fast Refresh сохраняет состояние useState
- Нет границы — полная перезагрузка

```
if (import.meta.hot) {
  import.meta.hot.accept((mod) => render(mod));
}
```

Vite+: CLI vp

- vp заменяет набор инструментов: dev, build, lint, fmt, test, run
- Конфиг остаётся vite.config.ts — Vite+ его расширяет
- vp add/remove/update, vpx (аналог npx)
- vp run <script> запускает скрипт из package.json

```
staged: { "*": "vp check --fix" },
run: { cache: { scripts: false, tasks: true } },
```

vp check = форматирование + линтинг + проверка типов за один проход

Vite+: хуки, задачи, кеш

- vp hooks enable/disable/status; скрипты в .vite-hooks
- pre-commit: vp staged — проверка только изменённых файлов
- run.tasks: именованные задачи с dependsOn и кешированием
- vp upgrade обновляет глобальный vp; CI — voidzero-dev/setup-vp

```
run: {
  tasks: {
    build: { command: 'vp build', dependsOn: ['lint'] },
  },
}
```

Oxc: Oxidation Compiler

- Набор высокопроизводительных инструментов для JS/TS на Rust
- Часть видения VoidZero: единый toolchain
- Включает: Oxlint, Oxfmt, Parser, Transformer, Minifier, Resolver, TS Runner
- Oxc питает Rolldown — бандлер Vite 8

> Oxidation — химический процесс, создающий ржавчину (rust)

Oxlint vs ESLint

	Oxlint	ESLint
Скорость	очень высокая (Rust)	средняя (JS)
Совместимость	целится в ESLint-экосистему	эталон
Конфиг	oxlint.config.ts / .oxlintrc.json	.eslintrc.*
Типизация	tsgo = TypeScript 7	tsc

- oxlint --init генерирует конфиг
- npm install -D oxlint; скрипты: "lint": "oxlint", "lint:fix": "oxlint --fix"
- Плагин eslint включён по умолчанию

Oxfmt: форматтер

- Совместим с Prettier, работает быстрее (Rust)
- Сортировка импортов, embedded-форматирование
- Игнор-файлы и inline-комментарии
- Миграция: prettier → oxfmt

```
// vite.config.ts
fmt: { /* настройки oxfmt */ }
```

> vp fmt запускает oxfmt; vp check включает fmt + lint + типы

Инструменты Oxc

- Parser — самый быстрый JS/TS-парсер с AST
- Transformer — TS, JSX, modern JS lowering
- Minifier — minify для продакшена
- Resolver — разрешение модулей
- TypeScript Runner — запуск TS/JSX в Node

> Все инструменты делят общие компоненты — меньше дублирования

Vite+ использует Oxc

```
// vite.config.ts
lint: {
  plugins: ["react", "typescript", "oxc"],
  rules: { "react/rules-of-hooks": "error" },
  options: { typeAware: true, typeCheck: true },
},
fmt: {},
staged: { "*": "vp check --fix" },
```

- vp lint = Oxlint; vp fmt = Oxfmt; vp check = всё вместе
- Конфиг линтера/форматтера — в vite.config.ts, а не в oxlint.config.ts
- JS-плагины и type-aware правила поддерживаются

TanStack Router: зачем

- 100% выведенный TypeScript: пути, params, search, контекст
- Типобезопасная навигация: Link, useNavigate, useParams, useSearch
- Встроенный SWR-кеш для loader'ов
- JSON-first search params — состояние в URL
- Файловая и кодовая маршрутизация одновременно

> URL — это state manager, а не просто строка

Типобезопасность

```
const router = createRouter({ routeTree });
declare module '@tanstack/react-router' {
  interface Register { router: typeof router }
}
```

- Register связывает типы роутера с экспортированными хуками
- Route.useParams() / Route.useSearch() — типизированы
- useNavigate({ from: Route.fullPath }) — подсказка контекста
- strict: false — для общих компонентов без привязки к роуту

Файловая vs кодовая маршрутизация

	Файловая	Кодовая
Определение	по файловой структуре	createRoute + addChildren
Типобезопасность	максимальная (генерация)	зависит от дисциплины
Когда	рекомендуется по умолчанию	полный контроль, динамика

- Оба подхода можно смешивать в одном проекте
- Файловая генерирует routeTree.gen.ts автоматически (Vite plugin)

Именование файлов маршрутов

Паттерн	Что значит
index.tsx	индексный маршрут родителя
$param.tsx	динамический сегмент
$.tsx	splat / catch-all
{-$param}.tsx	опциональный сегмент
_layout.tsx	pathless layout
posts_.$id.edit.tsx	non-nested (свой путь)
-file.tsx / -dir/	исключить из роутов (колокация)

Группы: (app)/, (auth)/ — чисто организационные папки

Кодовая маршрутизация: пример

```
const rootRoute = createRootRoute();
const postsRoute = createRoute({
  getParentRoute: () => rootRoute,
  path: 'posts',
});
const postRoute = createRoute({
  getParentRoute: () => postsRoute,
  path: '$postId',
  loader: ({ params }) => fetchPost(params.postId),
});
const routeTree = rootRoute.addChildren([
  postsRoute.addChildren([postRoute]),
]);
```

- getParentRoute обязателен — ради типобезопасности
- addChildren строит дерево вручную
- В реальных проектах смешивают с файловой

Параметры пути

```
// posts/$postId.tsx
export const Route = createFileRoute('/posts/$postId')({
  loader: ({ params }) => fetchPost(params.postId),
  component: PostComponent,
});
function PostComponent() {
  const { postId } = Route.useParams();
}
```

- $param — обязательный сегмент
- {-$param} — опциональный (undefined, если нет)
- $ — splat, значение в params._splat
- params доступны в loader, beforeLoad, компоненте

Layout и pathless routes

```
// app.tsx — layout route
export const Route = createFileRoute('/app')({
  component: () => (<div><h1>App</h1><Outlet /></div>),
});
// _pathlessLayout.tsx — без пути в URL
export const Route = createFileRoute('/_pathlessLayout')({
  component: () => (<div><h1>PL</h1><Outlet /></div>),
});
```

- Layout оборачивает дочерние роуты через <Outlet />
- Pathless (_prefix) оборачивает без сегмента в URL
- Non-nested: суффикс _ у родителя «отсоединяет» дочерний маршрут

Search params: состояние в URL

- JSON-first: вложенные структуры сериализуются автоматически
- Наследуются от родительских маршрутов
- Доступны в loader'ах, компонентах, хуках
- Обновление через Link search, useNavigate, <Navigate>

```
const { page, filter, sort } = Route.useSearch();
<Link from={Route.fullPath} search={(prev) => ({ page: prev.page + 1 })}>Next</Link>
```

Валидация search params: Zod

```
const schema = z.object({
  page: z.number().catch(1),
  filter: z.string().catch(''),
  sort: z.enum(['newest','oldest','price']).catch('newest'),
});
export const Route = createFileRoute('/shop/products')({
  validateSearch: zodValidator(schema),
});
```

- zodValidator из @tanstack/zod-adapter для Zod v3
- В Zod v4 схему можно передать напрямую
- fallback() сохраняет типы при ошибке валидации
- Поддерживаются Valibot, ArkType, Effect/Schema

Search middlewares

```
search: {
  middlewares: [
    retainSearchParams(['rootValue']),
    stripSearchParams({ page: 1 }),
  ],
}
```

- retainSearchParams — сохранить param во всех ссылках
- stripSearchParams — убрать param, если значение по умолчанию
- Можно цепочкой комбинировать

> По умолчанию search params обновляются независимо от pathname

Data loading: loader

```
export const Route = createFileRoute('/posts')({
  loader: ({ params, context }) => fetchPosts(),
});
// в компоненте:
const posts = Route.useLoaderData();
```

- loader вызывается при совпадении маршрута
- useLoaderData — типизированный результат
- beforeLoad — перед loader, для контекста и редиректов
- loaderDeps — ключ кеша по search params

Кеширование loader'ов (SWR)

- staleTime — свежесть данных (по умолч. 0 для nav, 30s для preload)
- gcTime — время жизни в кеше (по умолч. 5 мин)
- shouldReload — перезагружать ли при повторном входе
- staleReloadMode: 'background' | 'blocking'
- loaderDeps только из используемых params (не весь search!)

Контекст и beforeLoad (auth)

```
export const Route = createFileRoute('/posts')({
  beforeLoad: ({ context }) => {
    if (!context.auth.isAuthenticated) throw redirect({ to: '/login' });
    return { fetchPosts: () => ... };
  },
  loader: ({ context: { fetchPosts } }) => fetchPosts(),
});
```

- createRootRouteWithContext<T>() задаёт тип контекста
- Контекст наследуется по дереву маршрутов
- beforeLoad — идеальное место для проверки auth и редиректов

Pending и обработка ошибок

- pendingComponent показывается после pendingMs (по умолч. 1с)
- pendingMinMs (500мс) избегает мигания
- errorComponent с error и reset
- onError / onCatch для логирования
- AbortController в loader для отмены запросов

Preloading и code splitting

- preload стратегии: intent, viewport, render
- Компоненты роутов автоматически code-split'ятся
- lazy routes для тяжёлых страниц
- getRouteApi для доступа к Route без импорта конфига

```
// vite.config.ts — включить code splitting
// (по умолчанию в file-based routing)
```

Интеграция с TanStack Query

```
loader: ({ context: { queryClient }, params }) =>
  queryClient.ensureQueryData(postQueryOptions(params.postId)),
```

- router cache vs TanStack Query — выбирайте по размеру приложения
- defaultPreloadStaleTime: 0 чтобы внешний кеш решал
- shouldReload для условной перезагрузки
- QueryClient передаётся через контекст роутера

SSR и TanStack Start

- TanStack Router — client-first, Start добавляет сервер
- SSR, streaming, typed server functions
- Middleware, deployable server output
- Тот же route tree и типы — без переписывания

> Start использует тот же route tree — миграция из client-only прозрачна

Стек целиком

Слой	Инструмент
Сборка	Vite (dev: ESM, build: Rollup/Rolldown)
Линтинг / форматирование	Oxc: Oxlint + Oxfmt (Rust)
Роутинг	TanStack Router (типобезопасный)

- Vite+ объединяет всё: vp dev/build/lint/fmt/test
- Один конфиг vite.config.ts на весь стек
- TanStack Query — для серверного состояния поверх роутера

Что дальше

- Практика: свой проект на Vite+ с TanStack Router
- Изучить deferred data loading и route masking
- Настроить CI: voidzero-dev/setup-vp + vp check
- Документация: vite.dev, oxc.rs, tanstack.com/router

> Ссылки на доки и сообщество: discord.gg/artcraft